🔗 Chains in LangChain

---

2. What are Chains?

- **Problem**: LLM apps have multiple small steps (get input → send to LLM → process output → show to user).
  Doing all manually = tedious.
- **Chains** = A way to **connect multiple steps** into a **pipeline**.
- Benefits:
  - Automatic passing of outputs → next step's input.
  - Less boilerplate code (no need to manually call each .invoke()).
  - Easy to visualize.
  - Can support **sequential**, **parallel**, and **conditional** workflows.

👉 Think of Chains as **assembly lines** in a factory. Each step's product goes to the next step automatically.

---

3. Types of Chains

1. **Sequential Chains** → Steps run one after another.
2. **Parallel Chains** → Multiple branches run simultaneously.
3. **Conditional Chains** → Path chosen based on condition (like if/else).

---

4. Building Chains (with Examples)

🔹 **4.1 Simple Sequential Chain**

Steps:

1. Ask user for topic.
2. Send prompt to LLM → generate facts.
3. Parse response → return clean text.

Code Example:

```python
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain_core.output_parsers import StringOutputParser

# Step 1: Prompt
prompt = PromptTemplate.from_template(
    "Generate 5 interesting facts about {topic}"
)

# Step 2: Model
model = ChatOpenAI()

# Step 3: Parser
parser = StringOutputParser()

# Chain using | operator (LCEL syntax)
chain = prompt | model | parser

# Run chain
result = chain.invoke({"topic": "Cricket"})
print(result)
```

✅ Output → neat list of 5 facts.
✅ You can also **visualize chain**:
chain.get_graph().print_ascii()

🔹 4.2 Longer Sequential Chain (Two LLM Calls)

App flow:

1. User gives topic (e.g., *Unemployment in India*).
2. LLM generates **detailed report**.
3. Send that report back to LLM → summarize into **5 points**.

👉 Same idea, but two calls chained sequentially.
✅ Use PromptTemplate, ChatOpenAI, StringOutputParser.
✅ Connect them: prompt1 | model | parser | prompt2 | model | parser.

🔹 4.3 Parallel Chains

App: User gives a long document → we want:

- Notes
- Quiz questions

And then merge both into one output.

Steps:

1. Two parallel pipelines:
    - Prompt → Model1 → Parser → Notes
    - Prompt → Model2 → Parser → Quiz
2. Merge outputs with third model.

✅ Done with **RunnableParallel**.
from langchain.schema import RunnableParallel

- Lets you run multiple chains at the same time.
- Output is a dictionary like:

```
{
  "notes": "...",
  "quiz": "..."
}
```

Then merge both via another prompt.

🔹 4.4 Conditional Chains

App: Sentiment-based replies.

1. User gives feedback → classify into Positive/Negative.
2. If Positive → return "Thank you message."
3. If Negative → return "Sorry message."

✅ Done with **RunnableBranch** (like if/else).

To ensure consistent output (only "positive" or "negative"), we use **PydanticOutputParser** with a schema:

```
class Feedback(BaseModel):
    sentiment: Literal["positive", "negative"]
```

✅ This ensures the LLM always returns structured output.

✅ Then RunnableBranch chooses correct path.

---

5. Why Chains are Powerful?

- **Abstraction** → You don't need to write manual boilerplate.
- **Automation** → Steps auto-connect.
- **Scalability** → Easy to extend pipelines.
- **Flexibility** → Sequential, Parallel, Conditional flows.
- **Visualization** → Debug & explain pipelines easily.

---

6. Summary

- **Chains = pipelines** to connect multiple steps in LLM apps.
- Types:
  - Sequential
  - Parallel
  - Conditional
- Syntax: prompt | model | parser (LangChain Expression Language).
- Tools: RunnableParallel, RunnableBranch, RunnableLambda.
- Always combine with **Output Parsers** for clean, structured results.